TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL

Anexo de Evidencias

Metodología de Sistemas II

Trabajo Práctico Integrador

Integrantes:

- Grela, Ariel
- Higa, Matías

Comisión N° 1

M25 C4-06

Fecha: 08/11/2026

Tabla de Contenidos

1. Capturas de Pantalla 📷 5
CAP-01 5
CAP-02 5
CAP-03 6
CAP-04 6
CAP-05 7
CAP-06 8
CAP-07 9
CAP-08 10
CAP-09 10
CAP-10 11
CAP-11 11
CAP-12 12
CAP-13 12
CAP-14 13
CAP-15 13
CAP-16 13
CAP-17 14
CAP-18 14
CAP-19 14
CAP-20 14
CAP-21 15
CAP-22 15
CAP-23 15
CAP-24 15
CAP-25 16
CAP-26 16
CAP-27 17
CAP-28 17
CAP-29 18
CAP-30 18
CAP-31 18
CAP-32 19
CAP-33 19
CAP-34 20
CAP-35 20
CAP-36 20
CAP-37 21
CAP-38 21
CAP-39 21
CAP-40 21
CAP-41 22
CAP-42 22
CAP-43 22
CAP-44 22
CAP-45 23

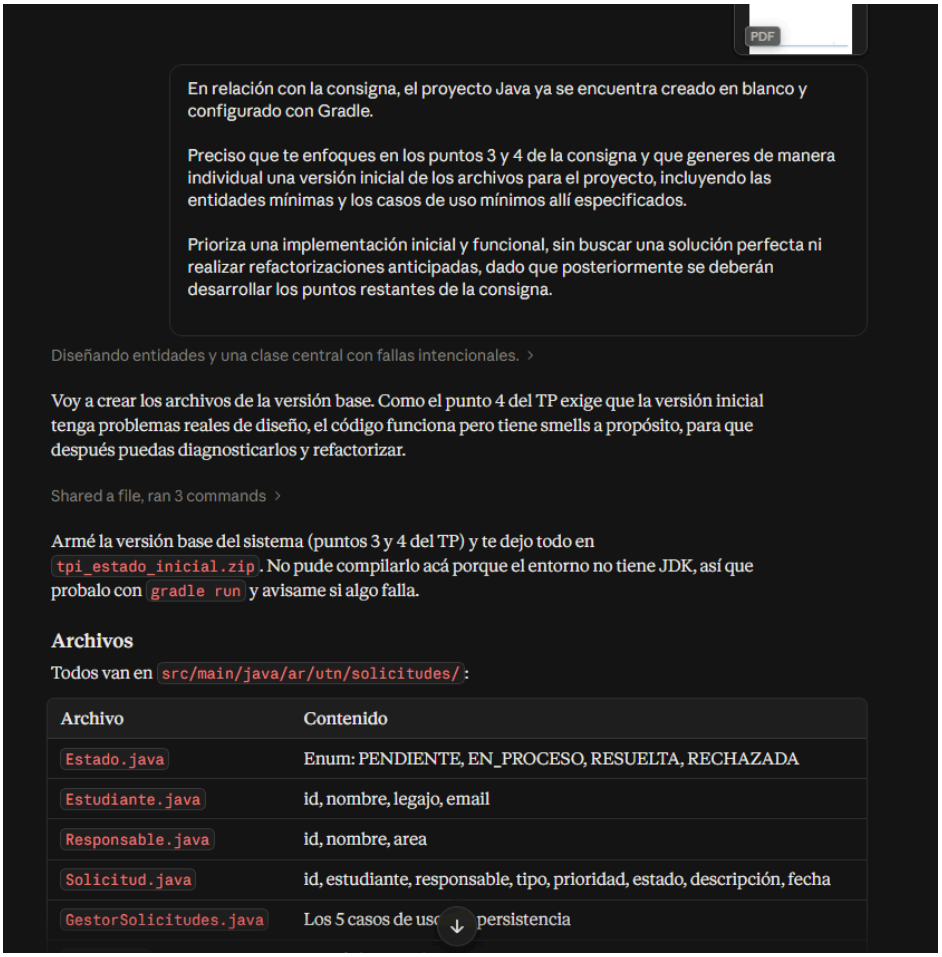
CAP-46 24
CAP-47 25
CAP-48 25
CAP-49 25
CAP-50 25
CAP-51 25
CAP-52 26
CAP-53 26
CAP-54 26
CAP-55 27
CAP-56 27
CAP-57 28
CAP-58 29
CAP-59 29
CAP-60 31
CAP-61 31
CAP-62 32
CAP-63 33
CAP-64 33
CAP-65 34
CAP-66 34
CAP-67 35
CAP-68 35
CAP-69 36
CAP-70 36
CAP-71 37
CAP-72 37
CAP-73 38
CAP-74 38
CAP-75 40
2. Uso de IA 🤖 41
2.1. Declaración general 41
2.2 Escala de grado de intervención 41
2.3 Registro por interacción 41
IA-01 41
IA-02 41
IA-03 42
IA-04 42
IA-05 42
IA-06 42
IA-07 43
IA-08 43
IA-09 43
IA-10 43
IA-11 44
IA-12 44
IA-13 44

IA-14.....44
IA-15.....45
IA-16.....45
3. Bibliografía y recursos consultados.....46

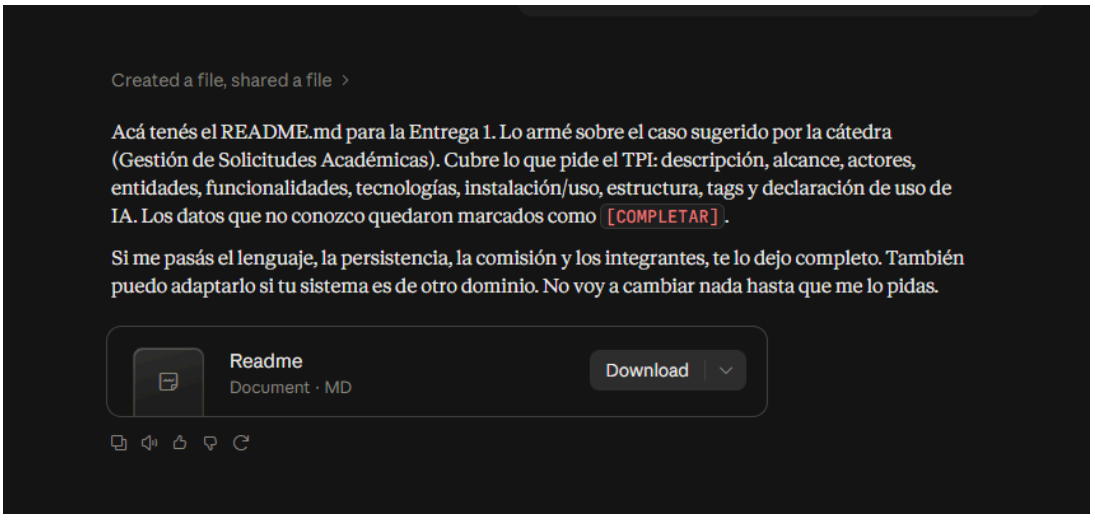
1. Capturas de Pantalla 📷

CAP-01 a CAP-58: etapas 1 a 3 y uso de IA de esas etapas. CAP-59 a CAP-75: code review (Etapa 7) y resultados finales.

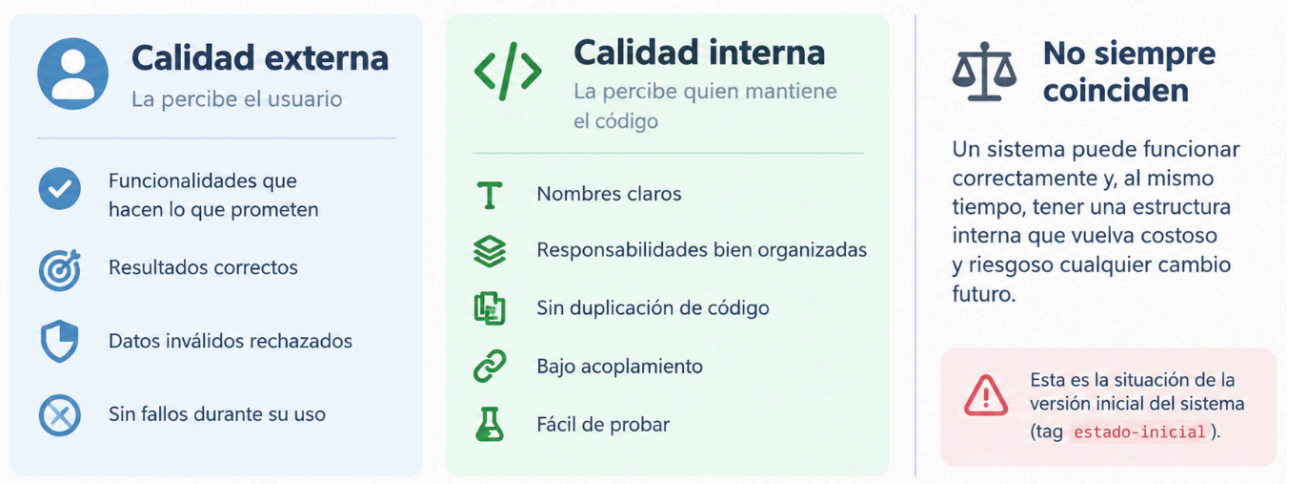
CAP-01



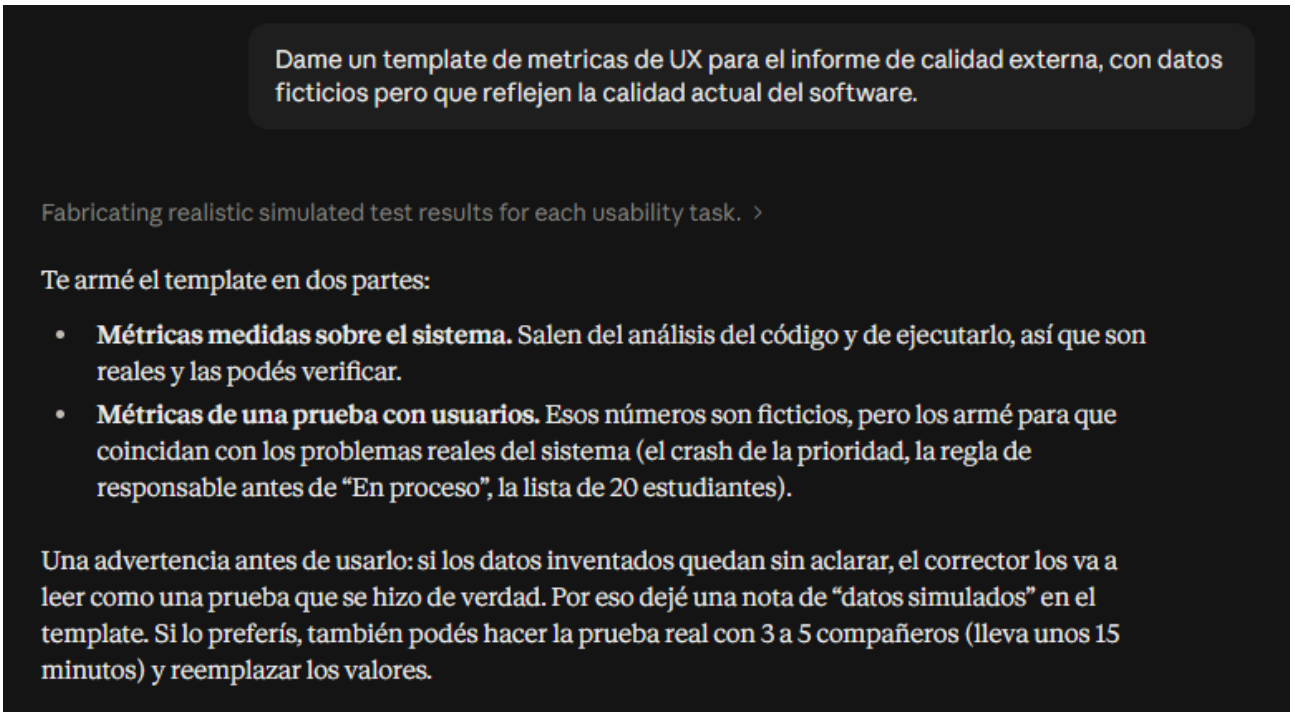
CAP-02



CAP-03


Calidad externa
La percibe el usuario
Funcionalidades que hacen lo que prometen
Resultados correctos
Datos inválidos rechazados
Sin fallos durante su uso
Calidad interna
La percibe quien mantiene el código
Nombres claros
Responsabilidades bien organizadas
Sin duplicación de código
Bajo acoplamiento
Fácil de probar
No siempre coinciden
Un sistema puede funcionar correctamente y, al mismo tiempo, tener una estructura interna que vuelva costoso y riesgoso cualquier cambio futuro.
Esta es la situación de la versión inicial del sistema (tag estado-inicial).

CAP-04


Dame un template de metricas de UX para el informe de calidad externa, con datos ficticios pero que reflejen la calidad actual del software.
Fabricating realistic simulated test results for each usability task. >
Te armé el template en dos partes:
• Métricas medidas sobre el sistema. Salen del análisis del código y de ejecutarlo, así que son reales y las podés verificar.
• Métricas de una prueba con usuarios. Esos números son ficticios, pero los armé para que coincidan con los problemas reales del sistema (el crash de la prioridad, la regla de responsable antes de "En proceso", la lista de 20 estudiantes).
Una advertencia antes de usarlo: si los datos inventados quedan sin aclarar, el corrector los va a leer como una prueba que se hizo de verdad. Por eso dejé una nota de "datos simulados" en el template. Si lo preferís, también podés hacer la prueba real con 3 a 5 compañeros (lleva unos 15 minutos) y reemplazar los valores.

CAP-05

```
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 18
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 3
Descripcion: Nota de parcial 1 de Gestión todavía no cargada en campus
Prioridad (1=alta, 2=media, 3=baja): 1
Solicitud registrada con id 4
```

CAP-06

```
Opcion: 1
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 11
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 4
Descripcion: hola
Prioridad (1=alta, 2=media, 3=baja): 1
Error: descripcion muy corta
```

CAP-07

```
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 5
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 2
Descripcion: Analisis Matematico -> Matematica II
Prioridad (1=alta, 2=media, 3=baja): 19
Error: prioridad invalida
```

CAP-08

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
La solicitud #1 no tiene responsable asignado. Solo puede rechazarse; para otros estados asignelo desde la opcion 3.
Nuevo estado:
1. RECHAZADA
Opcion:
```

CAP-09

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 2
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 2
Estado actualizado a RESUELTA
```

CAP-10

```
######################### GESTION DE SOLICITUDES ACADEMICAS #########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Error: la solicitud esta cerrada
```

CAP-11

```
######################### GESTION DE SOLICITUDES ACADEMICAS #########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 3
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 3
Responsable:
1. Maria Ortega S◆nchez (Alumnado)
2. Jorge Tchami (Alumnado)
3. Laura Gorosito (Secretaria Academica)
Opcion: 2
Responsable asignado: Jorge Tchami
```

CAP-12

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 4
Buscar por:
1. Estado
2. Tipo
3. Legajo
4. Prioridad
Opcion: 1
Estado:
1. PENDIENTE
2. EN_PROCESO
3. RESUELTA
4. RECHAZADA
Opcion: 2
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO
Resultados: 1
```

CAP-13

```
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 4
Buscar por:
1. Estado
2. Tipo
3. Legajo
4. Prioridad
Opcion: 3
Estudiante:
1. L1001 - Agustin Di Maria
2. L1002 - Lucia Kempes
3. L1003 - Tomas Maradona
4. L1004 - Valentina Passarella
5. L1005 - Nicolas Fillol
6. L1006 - Camila Batistuta
7. L1007 - Franco Houseman
8. L1008 - Martina Ardiles
9. L1009 - Santiago Bertoni
10. L1010 - Julieta Valdano
11. L1011 - Mateo Ruggeri
12. L1012 - Florencia Gallego
13. L1013 - Benjamin Tarantini
14. L1014 - Sofia Houseman
15. L1015 - Facundo Rattin
16. L1016 - Micaela Sivori
17. L1017 - Ignacio Labruna
18. L1018 - Paula Ayala
19. L1019 - Agustin Caniggia
20. L1020 - Julian Brown
Opcion: 1
#1 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | RECHAZADA
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO
Resultados: 2
```

CAP-14

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 5
===== REPORTE DE SOLICITUDES =====
#1 | 2026-10-22 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | RECHAZADA | SIN ASIGNAR
#2 | 2026-10-22 | Lucia Kempes | RECLAMO | Prioridad: 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | 2026-10-22 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO | Jorge Tchami (Alumnado)
#4 | 2026-10-22 | Paula Ayala | RECLAMO | Prioridad: 1 | PENDIENTE | SIN ASIGNAR
#5 | 2026-10-22 | Julieta Valdano | EQUIVALENCIA | Prioridad: 1 | PENDIENTE | SIN ASIGNAR
---------------------------------
Total: 5
Pendientes: 2 | En proceso: 1 | Resueltas: 1 | Rechazadas: 1
Alta: 3 | Media: 1 | Baja: 1
```

CAP-15

```
########################## GESTION DE SOLICITUDES ACADEMICAS ##########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 666
Opcion invalida
```

CAP-16

```
Execution failed for task ':tupad.ms2.Main.main()'.
> Process 'command 'C:\Users\matut\.jdks\ms-21.0.12.1\bin\java.exe'' finished with non-zero exit value 1

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights from a Build Scan (powered by Develocity).
> Get more help at https://help.gradle.org.
BUILD FAILED in 17m 39s
2 actionable tasks: 1 executed, 1 up-to-date
```

CAP-17

```
                String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                int t = elegir(sc, titulo: "Tipo:", tipos);
                if (t == -1) {
                    System.out.println("Registro cancelado.");
                } else {
                    System.out.print("Descripcion: ");
                    String d = sc.nextLine();
                    System.out.print("Prioridad (1=alta, 2=media, 3=baja): ");
                    int p = Integer.parseInt(sc.nextLine());
                    g.registrarSolicitud(est.legajo, tipos[t], d, p);
                }
            }
        }
```

CAP-18

```
public class Solicitud {  23 usages  Matias H
    public int id;  12 usages
    public Estudiante est;  5 usages
    public Responsable resp;  12 usages
    public String tipo;        // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5 usages
    public int prioridad;      // 1 = alta, 2 = media, 3 = baja  8 usages
    public Estado estado;  22 usages
    public String desc;  1 usage
    public String fecha;  2 usages
```

CAP-19

```
                String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                int t = elegir(sc, titulo: "Tipo:", tipos);
                if (t == -1) {
                    System.out.println("Registro cancelado.");
```

CAP-20

```
            } else if (c == 1) {
                String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                int sel = elegir(sc, titulo: "Tipo:", tipos);
                if (sel != -1) valor = tipos[sel];
```

CAP-21

```
140     // Asignar responsable (del listado) a solicitud
141     public boolean asignarResponsable(int id, int idResponsable) {  2 usages  Matias H
142         Solicitud s = null;
143         for (Solicitud x : solicitudes) {
144             if (x.id == id) {
145                 s = x;
146             }
147         }
```

CAP-22

```
178     // Precarga de solicitudes al iniciar, sin mostrar los mensajes del gestor (estudiantes y responsables ya vienen cargados desde GestorSolicitudes)
179 @   private static void cargarDatosEjemplo(GestorSolicitudes g) {  1 usage  Matias H *
180         PrintStream original = System.out;
181         System.setOut(new PrintStream(OutputStream.nullOutputStream()));
182         try {
183             g.registrarSolicitud( legajo: "L1001",  tipo: "CONSTANCIA",  desc: "Constancia de alumno regular",  prioridad: 2);
184             g.registrarSolicitud( legajo: "L1002",  tipo: "RECLAMO",  desc: "Error en nota del parcial",  prioridad: 1);
```

CAP-23

```
12      import java.util.Scanner;
13
14      public class Main {  Matias H
15          public static void main(String[] args) {...}
177
178         // Precarga de solicitudes al iniciar, sin mostr
```

CAP-24

```
Structure
GestorSolicitudes
    solicitudes: List<Solicitud> = new ArrayList<>()
    estudiantes: List<Estudiante> = new ArrayList<>()
    responsables: List<Responsable> = new ArrayList<>()
    contador: int = 1
    GestorSolicitudes()
    cargarEstudiantes(): void
    cargarResponsables(): void
    registrarSolicitud(String, String, String, int): int
    cambiarEstado(int, String): boolean
    asignarResponsable(int, int): boolean
    buscar(String, String): List<Solicitud>
    reporte(): void
```

CAP-25

GestorSolicitudes.java

```
9   public class GestorSolicitudes {  6 usages  Matias H
62      // Registrar solicitud
63      public int registrarSolicitud(String legajo, String tipo, String desc, int prioridad) {  4 usages  Matias H
64          Estudiante e = null;
65          for (Estudiante x : estudiantes) {
66              if (x.legajo.equals(legajo)) {
67                  e = x;
68              }
69          }
70          if (e == null) {
71              System.out.println("Error: estudiante no encontrado");
72              return -1;
73          }
74          if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
75                  && !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
76              System.out.println("Error: tipo invalido");
77              return -1;
78          }
79          if (prioridad < 1 || prioridad > 3) {
80              System.out.println("Error: prioridad invalida");
81              return -1;
82          }
83          if (desc == null || desc.length() < 5) {
84              System.out.println("Error: descripcion muy corta");
85              return -1;
86          }
```

CAP-26

```
63      public int registrarSolicitud(String legajo, String tipo, String desc, int prioridad) {
64          Estudiante e = null;
65          for (Estudiante x : estudiantes) {
66              if (x.legajo.equals(legajo)) {
67                  e = x;
68              }
69          }
70          if (e == null) {
71              System.out.println("Error: estudiante no encontrado");
72              return -1;
73          }
74          if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
75                  && !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
76              System.out.println("Error: tipo invalido");
77              return -1;
78          }
79          if (prioridad < 1 || prioridad > 3) {
80              System.out.println("Error: prioridad invalida");
81              return -1;
82          }
83          if (desc == null || desc.length() < 5) {
84              System.out.println("Error: descripcion muy corta");
85              return -1;
86          }
```

CAP-27

```java
// Cambiar estado de solicitud
public boolean cambiarEstado(int id, String nuevoEstado) {  2 usages  Matias H
    Solicitud s = null;
    for (Solicitud x : solicitudes) {
        if (x.id == id) {
            s = x;
        }
    }
    if (s == null) {
        System.out.println("Error: solicitud no encontrada");
        return false;
    }
    if (nuevoEstado.equals("EN_PROCESO")) {
        if (s.estado == Estado.PENDIENTE) {
            if (s.resp == null) {
                System.out.println("Error: debe asignar responsable primero");
                return false;
            }
            s.estado = Estado.EN_PROCESO;
        } else {
            System.out.println("Error: transicion invalida");
            return false;
        }
    } else if (nuevoEstado.equals("RESUELTA")) {
        if (s.estado == Estado.EN_PROCESO) {
            s.estado = Estado.RESUELTA;
        } else {
            System.out.println("Error: transicion invalida");
            return false;
        }
```

CAP-28

```java
// Asignar responsable (del listado) a solicitud
public boolean asignarResponsable(int id, int idResponsable) {  2 usages
    Solicitud s = null;
    for (Solicitud x : solicitudes) {
        if (x.id == id) {
            s = x;
        }
    }
    if (s == null) {
        System.out.println("Error: solicitud no encontrada");
        return false;
    }
    if (s.estado == Estado.RESUELTA || s.estado == Estado.RECHAZADA) {
        System.out.println("Error: la solicitud esta cerrada");
        return false;
    }
    Responsable r = null;
    for (Responsable x : responsables) {
        if (x.id == idResponsable) {
```

CAP-29

```
196                 }
197             }
198         } else {
199             System.out.println("Criterio no soportado");
200         }
201         return res;
```

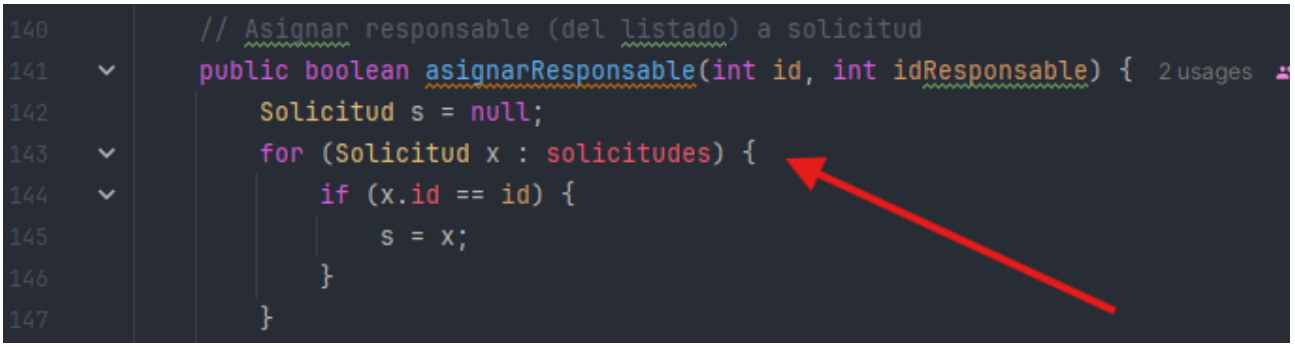
CAP-30

```
194 @   private static Solicitud pedirSolicitud(Scanner sc, GestorSolicitudes g)
195         System.out.print("Id solicitud: ");
196         try {
197             int id = Integer.parseInt(sc.nextLine().trim());
198             for (Solicitud x : g.solicitudes) {
199                 if (x.id == id) {
200                     return x;
201                 }
202             }
203         } catch (NumberFormatException ex) {
204             // cae al return null
205         }
206         return null;
207     }
```

CAP-31

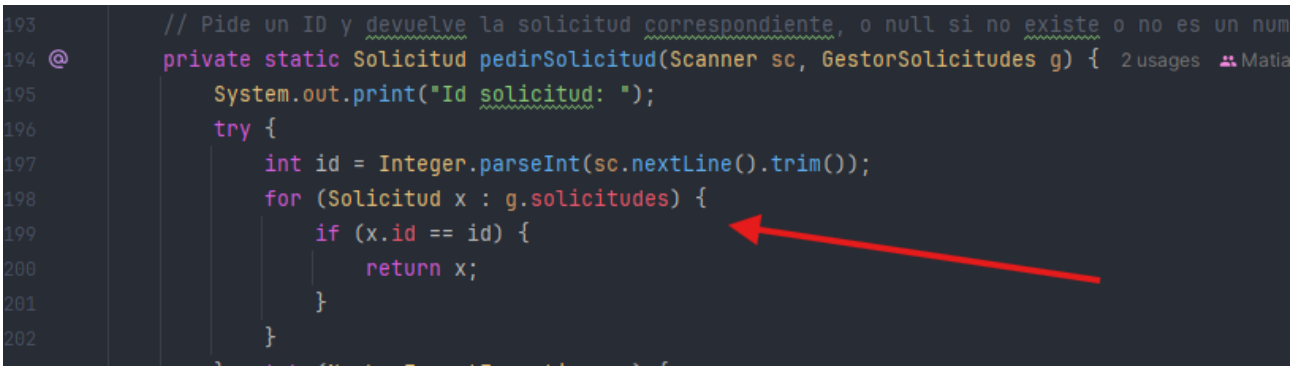
```
95      // Cambiar estado de solicitud
96      public boolean cambiarEstado(int id, String nuevoEstado) {
97          Solicitud s = null;
98          for (Solicitud x : solicitudes) {
99              if (x.id == id) {
100                 s = x;
101             }
102         }
103         if (s == null) {
```

CAP-32



```
140        // Asignar responsable (del listado) a solicitud
141        public boolean asignarResponsable(int id, int idResponsable) {  2 usages
142            Solicitud s = null;
143            for (Solicitud x : solicitudes) {
144                if (x.id == id) {
145                    s = x;
146                }
147            }
```

CAP-33



```
193        // Pide un ID y devuelve la solicitud correspondiente, o null si no existe o no es un num
194        private static Solicitud pedirSolicitud(Scanner sc, GestorSolicitudes g) {  2 usages  Matia
195            System.out.print("Id solicitud: ");
196            try {
197                int id = Integer.parseInt(sc.nextLine().trim());
198                for (Solicitud x : g.solicitudes) {
199                    if (x.id == id) {
200                        return x;
201                    }
202                }
```

CAP-34

```
                if (sol == null) {
                    System.out.println("Error: solicitud no encontrada");
                } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
                    System.out.println("Error: la solicitud esta cerrada");
                } else {
                    String[] estados;
                    if (sol.resp == null) {
                        System.out.println("La solicitud #" + sol.id + " no tiene responsable asignado. "
                                + "Solo puede rechazarse; para otros estados asignelo desde la opcion 3.");
                        estados = new String[]{"RECHAZADA"};
                    } else {
                        estados = new String[]{"EN_PROCESO", "RESUELTA", "RECHAZADA"};
                    }
                    int e = elegir(sc, titulo: "Nuevo estado:", estados);
                    if (e != -1) {
                        g.cambiarEstado(sol.id, estados[e]);
                    }
                }
            }

        } else if (op.equals("3")) {
            if (g.solicitudes.isEmpty()) {
                System.out.println("No hay solicitudes cargadas.");
            } else {
                listarSolicitudes(g);
                Solicitud sol = pedirSolicitud(sc, g);

                if (sol == null) {
                    System.out.println("Error: solicitud no encontrada");
                } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
                    System.out.println("Error: la solicitud esta cerrada");
                } else {
```

CAP-35

```
                } else {
                    Estudiante est = g.estudiantes.get(sEst);

                    String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                    int t = elegir(sc, titulo: "Tipo:", tipos);
                    if (t == -1) {
                        System.out.println("Registro cancelado.");
                    } else {
                        System.out.print("Descripcion: ");
```

CAP-36

```

                } else if (c == 1) {
                    String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                    int sel = elegir(sc, titulo: "Tipo:", tipos);
                    if (sel != -1) valor = tipos[sel];

                } else if (c == 2) {
                    if (g.estudiantes.isEmpty()) {
```

CAP-37

```
72              return -1;
73          }
74          if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
75                  && !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
76              System.out.println("Error: tipo invalido");
77              return -1;
78          }
79          if (prioridad < 1 || prioridad > 3) {
80              System.out.println("Error: prioridad invalida");
```

CAP-38

```
5   public class Solicitud {  23 usages  Matias H
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;         // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5
10      public int prioridad;       // 1 = alta, 2 = media, 3 = baja  8 usages
```

CAP-39

```
5   public class Solicitud {  23 usages  Matias H
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;         // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5
```

CAP-40

```
205     public void reporte() {  1 usage  Matias H
206         int pend = 0, proc = 0, res = 0, rech = 0;
207         int alta = 0, media = 0, baja = 0;
208         System.out.println("===== REPORTE DE SOLICITUDES =====");
209         for (Solicitud s : solicitudes) {
210             String r = (s.resp == null) ? "SIN ASIGNAR" : s.resp.nombre + " (" + s.resp.area + ")";
211             System.out.println("#" + s.id + " | " + s.fecha + " | " + s.est.nombre + " | " + s.tipo
212                     + " | Prioridad: " + s.prioridad + " | " + s.estado + " | " + r);
213             if (s.estado == Estado.PENDIENTE) pend++;
214             if (s.estado == Estado.EN_PROCESO) proc++;
215             if (s.estado == Estado.RESUELTA) res++;
216             if (s.estado == Estado.RECHAZADA) rech++;
217             if (s.prioridad == 1) alta++;
218             if (s.prioridad == 2) media++;
219             if (s.prioridad == 3) baja++;
220         }
```

CAP-41

```
54                      } else {
55                          System.out.print("Descripcion: ");
56                          String d = sc.nextLine();
57                          System.out.print("Prioridad (1=alta, 2=media, 3=baja): ");
58                          int p = Integer.parseInt(sc.nextLine());
59                          g.registrarSolicitud(est.legajo, tipos[t], d, p);
60                      }
```

CAP-42

```
86          }
87
88          Solicitud s = new Solicitud(contador, e, tipo, prioridad, desc, LocalDate.now().toString());
89          contador++;
90          solicitudes.add(s);
```

CAP-43

```
192         } else if (criterio.equals("prioridad")) {
193             for (Solicitud s : solicitudes) {
194                 if (String.valueOf(s.prioridad).equals(valor)) {
195                     res.add(s);
196                 }
197             }
```

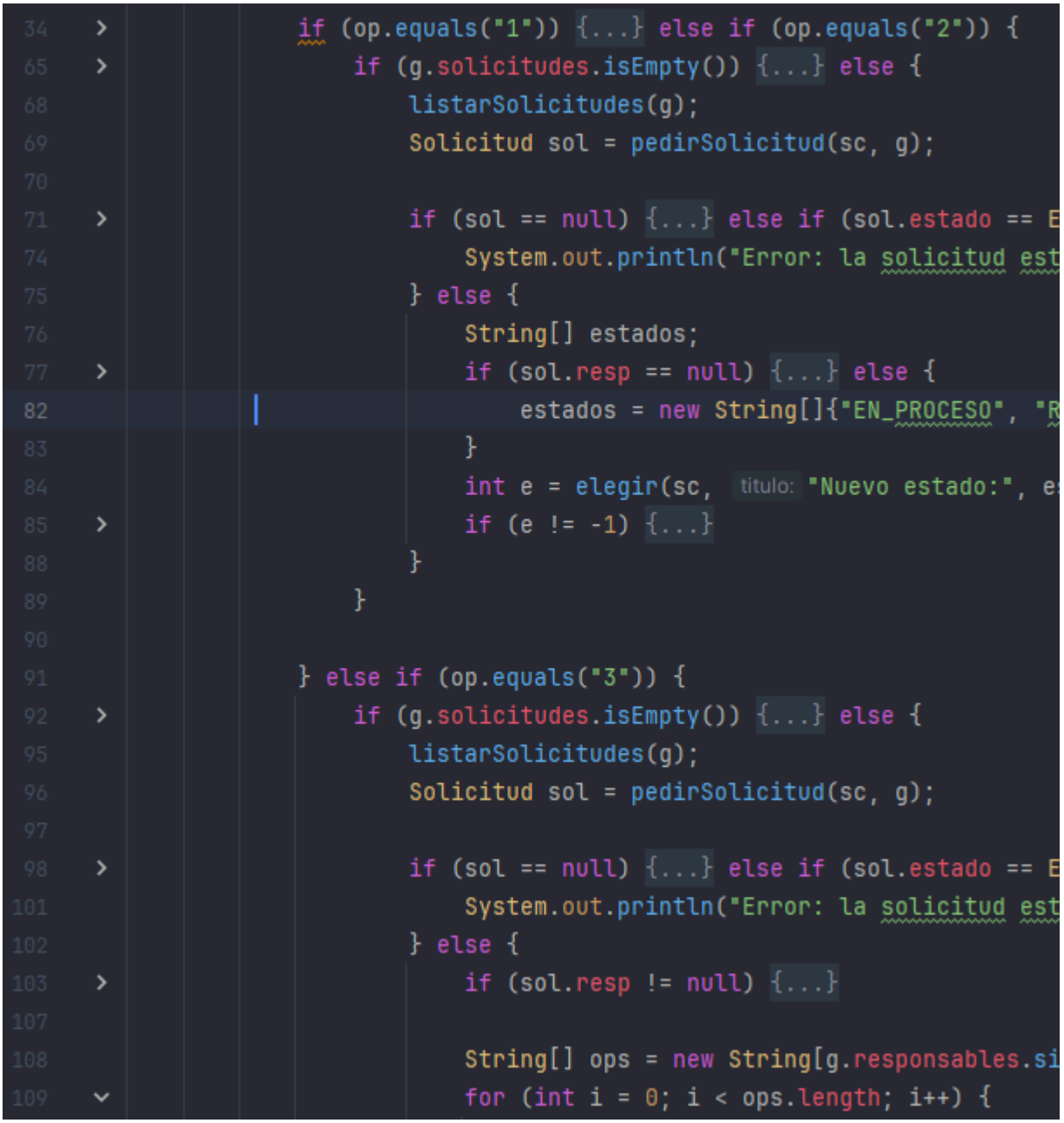
CAP-44

```
205     public void reporte() {  1 usage   Matias H
206         int pend = 0, proc = 0, res = 0, rech = 0;
207         int alta = 0, media = 0, baja = 0;
208         System.out.println("===== REPORTE DE SOLICITUDES ====
209         for (Solicitud s : solicitudes) {
210             String r = (s.resp == null) ? "SIN ASIGNAR" : s.
211             System.out.println("#" + s.id + " | " + s.fecha +
212                     + " | Prioridad: " + s.prioridad + " | "
213             if (s.estado == Estado.PENDIENTE) pend++;
214             if (s.estado == Estado.EN_PROCESO) proc++;
215             if (s.estado == Estado.RESUELTA) res++;
216             if (s.estado == Estado.RECHAZADA) rech++;
217             if (s.prioridad == 1) alta++;
218             if (s.prioridad == 2) media++;
219             if (s.prioridad == 3) baja++;
220         }
```

CAP-45

```
171        // Buscar / Filtrar
172 @      public List<Solicitud> buscar(String criterio, String valor) {  1 usage
173            List<Solicitud> res = new ArrayList<>();
174            if (criterio.equals("estado")) {
175                for (Solicitud s : solicitudes) {
176                    if (s.estado.toString().equalsIgnoreCase(valor)) {
177                        res.add(s);
178                    }
179                }
180            } else if (criterio.equals("tipo")) {
181                for (Solicitud s : solicitudes) {
182                    if (s.tipo.equalsIgnoreCase(valor)) {
183                        res.add(s);
184                    }
185                }
186            } else if (criterio.equals("legajo")) {
187                for (Solicitud s : solicitudes) {
188                    if (s.est.legajo.equals(valor)) {
189                        res.add(s);
190                    }
191                }
192            } else if (criterio.equals("prioridad")) {
193                for (Solicitud s : solicitudes) {
194                    if (String.valueOf(s.prioridad).equals(valor)) {
195                        res.add(s);
196                    }
197                }
198            } else {
199                System.out.println("Criterio no soportado");
200            }
201            return res;
```

CAP-46


if (op.equals("1")) {...} else if (op.equals("2")) {
if (g.solicitudes.isEmpty()) {...} else {
listarSolicitudes(g);
Solicitud sol = pedirSolicitud(sc, g);
if (sol == null) {...} else if (sol.estado == E
System.out.println("Error: la solicitud est
} else {
String[] estados;
if (sol.resp == null) {...} else {
estados = new String[]{"EN_PROCESO", "R
}
int e = elegir(sc, titulo: "Nuevo estado:", e
if (e != -1) {...}
}
}
} else if (op.equals("3")) {
if (g.solicitudes.isEmpty()) {...} else {
listarSolicitudes(g);
Solicitud sol = pedirSolicitud(sc, g);
if (sol == null) {...} else if (sol.estado == E
System.out.println("Error: la solicitud est
} else {
if (sol.resp != null) {...}
String[] ops = new String[g.responsables.si
for (int i = 0; i < ops.length; i++) {

CAP-47

```
4
5   public class Solicitud {  23 usages  Matias
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;          // CONSTANC
10      public int prioridad;        // 1 = alta
11      public Estado estado;  23 usages
12      public String desc;  1 usage
13      public String fecha;  2 usages
```

CAP-48

```
171     // Buscar / Filtrar
172 @   public List<Solicitud> buscar(String criterio, String valor) {
173         List<Solicitud> res = new ArrayList<>();
```

CAP-49

```
64              } else if (op.equals("2")) {
65                  if (g.solicitudes.isEmpty()) {
66                      System.out.println("No hay so
```

CAP-50

```
107             if (nuevoEstado.equals("EN_PROCESO")) {
108                 if (s.estado == Estado.PENDIENTE) {
109                     if (s.resp == null) {
110                         System.out.println("Error: deb
111                         return false;
112                     }
113                     s.estado = Estado.EN_PROCESO;
114                 } else {
```

CAP-51

```
71              if (sol == null) {
72                  System.out.println("Error: solicitud no encontrada");
73              } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
74                  System.out.println("Error: la solicitud esta cerrada");
75              } else {
```

CAP-52

```
122                } else if (op.equals("4")) {
123                    String[] criterios = {"estado", "tipo", "legajo", "pr
124                    int c = elegir(sc, titulo: "Buscar por:", new String[]{
125                    String valor = null;
126
127                    if (c == 0) {
128                        Estado[] es = Estado.values();
129                        String[] nombres = new String[es.length];
130                        for (int i = 0; i < es.length; i++) {
131                            nombres[i] = es[i].toString();
132                        }
133                        int sel = elegir(sc, titulo: "Estado:", nombres);
134                        if (sel != -1) valor = nombres[sel];
135
```

CAP-53

```
 9      public class GestorSolicitudes {  6 usages  Matias H
10
11          public List<Solicitud> solicitudes = new ArrayList<>();
12          public List<Estudiante> estudiantes = new ArrayList<>();
13          public List<Responsable> responsables = new ArrayList<>();
```

CAP-54

```
178     // Precarga de solicitudes al iniciar, sin mostrar los mensajes del gestor (estudiantes y responsables ya vienen ca
179 @   private static void cargarDatosEjemplo(GestorSolicitudes g) {  1 usage  Matias H
180         PrintStream original = System.out;
181         System.setOut(new PrintStream(OutputStream.nullOutputStream()));
182         try {
183             g.registrarSolicitud( legajo: "L1001",  tipo: "CONSTANCIA",  desc: "Constancia de alumno regular",  prioridad: 2);
184             g.registrarSolicitud( legajo: "L1002",  tipo: "RECLAMO",  desc: "Error en nota del parcial",  prioridad: 1);
185             g.registrarSolicitud( legajo: "L1001",  tipo: "EQUIVALENCIA",  desc: "Equivalencia de Matematica I",  prioridad: 3);
186             g.asignarResponsable( id: 2,  idResponsable: 1);   // Maria Lopez (Alumnado)
187             g.cambiarEstado( id: 2,  nuevoEstado: "EN_PROCESO");
188         } finally {
189             System.setOut(original);
190         }
191     }
```

CAP-55

```java
public Solicitud(int id, Estudiante est, String tipo, int prioridad, String desc, String fecha)
    this.id = id;
    this.est = est;
    this.tipo = tipo;
    this.prioridad = prioridad;
    this.desc = desc;
    this.fecha = fecha;
    this.estado = Estado.PENDIENTE;
}
```

CAP-56

Etapa 3: Deuda técnica y code smells

3.1 Criterio de diagnóstico

Se revisó el código de `estado-inicial` buscando los code smells del catálogo de Fowler y debilidades técnicas generales. Cada hallazgo se registra con:

- **Evidencia:** qué se observa en el código.
- **Ubicación:** archivo y líneas.
- **Impacto:** qué consecuencia tiene hoy sobre la legibilidad, la mantenibilidad o la evolución.
- **Riesgo de mantenimiento:** qué puede salir mal al modificar el código.
- **Prioridad de corrección:**
  - **Alta:** causa defectos visibles, bloquea otras mejoras (por ejemplo las pruebas) o concentra la mayor parte de los cambios futuros.
  - **Media:** dificulta el mantenimiento pero no rompe nada por sí solo.
  - **Baja:** afecta la prolijidad y su corrección es barata y sin riesgo.

La deuda de la versión inicial es **deliberada y prudente**: el sistema se construyó sin criterios de diseño a propósito, para tener un punto de partida diagnosticable. Su "interés" es el costo extra que tiene cada cambio mientras la deuda siga ahí.

3.2 Resumen de hallazgos

ID	Code smell / debilidad	Ubicación principal	Prioridad
CS-01	Long Method	`Main.main()` (15–176)	Alta
CS-02	Large Class / God Class	`GestorSolicitudes`	Alta
CS-03	Lógica de negocio acoplada a la consola	`GestorSolicitudes` (23 `System.out`), `Main.cargarDatosEjemplo()`	Alta
CS-04	Manejo de errores inconsistente	`GestorSolicitudes`, `Main` (58, 203–205, 221–223)	Alta
CS-05	Duplicated Code	`Main` y `GestorSolicitudes` (varios)	Alta
CS-06	Shotgun Surgery	Tipos y estados dispersos en 4 archivos	Alta

CAP-57

es realmente necesario señalar todas las lineas de código en todos los casos dentro de cada code smell?

No. La consigna pide **ubicación** para cada smell, no un inventario de todas las líneas. Con archivo, método y un rango de líneas, o la línea más representativa, alcanza para que el docente lo encuentre. Listar todo agrega ruido y hace más fácil equivocarse con un número de línea.

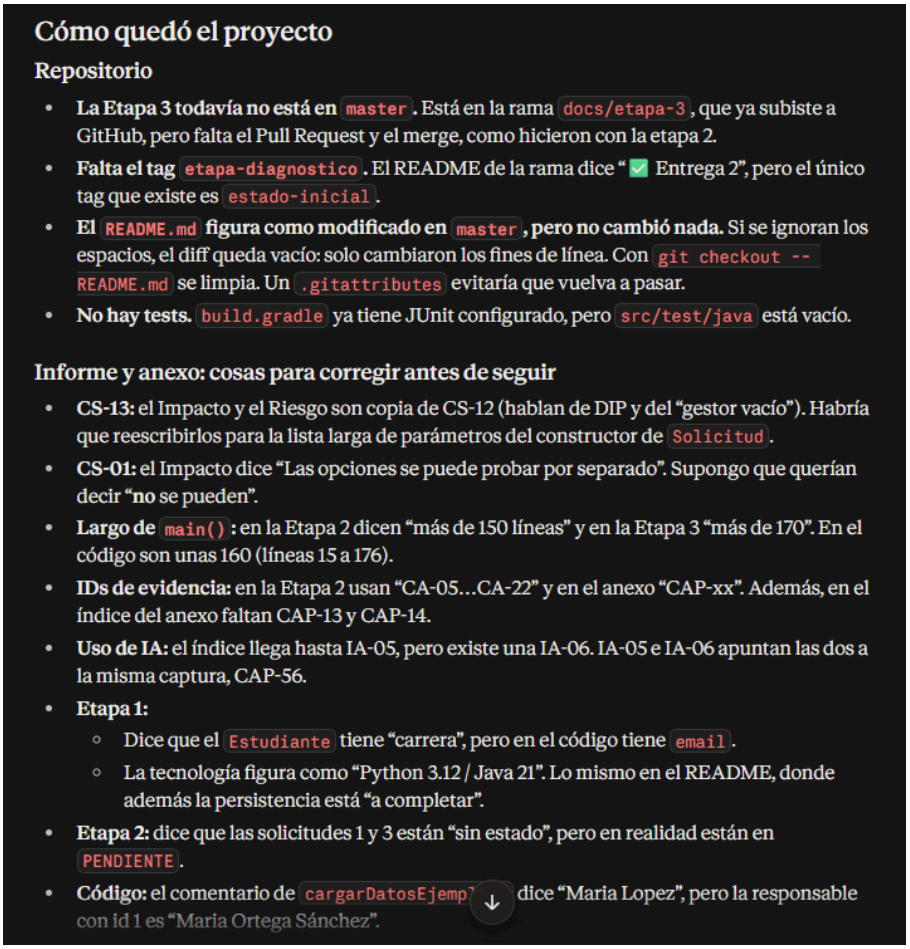
Las líneas exactas solo valen la pena cuando la cantidad de lugares es justamente la evidencia:

- **CS-05 Duplicated Code:** hay que mostrar que la misma lógica aparece en varios sitios. Ahí sí conviene la tabla con las ubicaciones, aunque pueden quedarse con 3 o 4 filas en vez de 6.
- **CS-06 Shotgun Surgery:** el punto es cuántos lugares hay que tocar para un cambio chico. Igual pueden resumirlo así: "agregar un tipo obliga a modificar 4 lugares en 3 archivos (`Main`, `GestorSolicitudes`, `Solicitud`)", sin detallar cada línea. La captura de la búsqueda ya muestra el resto.

Para los demás, una ubicación resumida es suficiente:

Smell	Ubicación suficiente
CS-01	`Main.main()`, líneas 15–176
CS-02	`GestorSolicitudes` (clase completa)
CS-03	`GestorSolicitudes` (23 `System.out`, todos sus métodos) y `Main.cargarDatosEjemplo()`
CS-04	`Main`, línea 58; métodos públicos de `GestorSolicitudes`
CS-07	`Solicitud`: campos `tipo`, `prioridad`, `fecha`
CS-08	`GestorSolicitudes.buscar()` y `cambiarEstado()`

CAP-58



Cómo quedó el proyecto

Repositorio

- **La Etapa 3 todavía no está en** `master`. Está en la rama `docs/etapa-3`, que ya subiste a GitHub, pero falta el Pull Request y el merge, como hicieron con la etapa 2.
- **Falta el tag** `etapa-diagnostico`. El README de la rama dice "✅ Entrega 2", pero el único tag que existe es `estado-inicial`.
- **El** `README.md` **figura como modificado en** `master`**, pero no cambió nada.** Si se ignoran los espacios, el diff queda vacío: solo cambiaron los fines de línea. Con `git checkout -- README.md` se limpia. Un `.gitattributes` evitaría que vuelva a pasar.
- **No hay tests.** `build.gradle` ya tiene JUnit configurado, pero `src/test/java` está vacío.

Informe y anexo: cosas para corregir antes de seguir

- **CS-13:** el Impacto y el Riesgo son copia de CS-12 (hablan de DIP y del "gestor vacío"). Habría que reescribirlos para la lista larga de parámetros del constructor de `Solicitud`.
- **CS-01:** el Impacto dice "Las opciones se puede probar por separado". Supongo que querían decir "**no** se pueden".
- **Largo de** `main()`**:** en la Etapa 2 dicen "más de 150 líneas" y en la Etapa 3 "más de 170". En el código son unas 160 (líneas 15 a 176).
- **IDs de evidencia:** en la Etapa 2 usan "CA-05…CA-22" y en el anexo "CAP-xx". Además, en el índice del anexo faltan CAP-13 y CAP-14.
- **Uso de IA:** el índice llega hasta IA-05, pero existe una IA-06. IA-05 e IA-06 apuntan las dos a la misma captura, CAP-56.
- **Etapa 1:**
  - Dice que el `Estudiante` tiene "carrera", pero en el código tiene `email`.
  - La tecnología figura como "Python 3.12 / Java 21". Lo mismo en el README, donde además la persistencia está "a completar".
- **Etapa 2:** dice que las solicitudes 1 y 3 están "sin estado", pero en realidad están en `PENDIENTE`.
- **Código:** el comentario de `cargarDatosEjemp` dice "Maria Lopez", pero la responsable con id 1 es "Maria Ortega Sánchez".

Las capturas CAP-59 a CAP-74 se generaron a partir del código y de ejecuciones reales de cada versión: el "antes" corresponde al commit `671290b` (previo a las correcciones) y el "después" al `70fb575`. En las de consola, lo que tipea el usuario se muestra en amarillo. CAP-72 y CAP-73 son los reportes HTML de JaCoCo y de Gradle. CAP-75 es la inspección de IntelliJ que tomó Matías.

CAP-59

CR-01 — antes: con la solicitud #1 pendiente y con responsable, el menú ofrecía RESUELTA (rechazada con "transicion invalida") y, ya en proceso, volvía a ofrecer EN_PROCESO.

CR-01 - antes: el menú ofrecía transiciones que después rechazaba — java tupad.ms2.Main (lo tipeado por el usuario, en amarillo)

```
Responsable asignado: Laura Gorosito

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | Laura Gorosito (Secretaria Academica)
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega Sánchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 2
Error: transicion invalida

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | Laura Gorosito (Secretaria Academica)
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega Sánchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 1
Estado actualizado a EN_PROCESO

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | EN_PROCESO | Laura Gorosito (Secretaria Academica)
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega Sánchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 1
Error: transicion invalida
```

CAP-60

CR-01 — después: `Estado.requiereResponsable()`, `Solicitud.estadosPosibles()` y `MenuConsola.cambiarEstado()`, que solo muestra esa lista.

CR-01 · las opciones salen del dominio — Estado.java · Solicitud.java · MenuConsola.java

```
34      /** Para trabajar una solicitud alguien tiene que hacerse cargo: EN_PROCESO exige responsable. */
35      public boolean requiereResponsable() {
36          return this == EN_PROCESO;
37      }
        // ...
50      /**
51       * Estados a los que puede pasar ahora, según el ciclo de vida y si tiene responsable.
52       * La interfaz ofrece solo estos, así no muestra opciones que el dominio después rechaza.
53       */
54      public List<Estado> estadosPosibles() {
55          return Arrays.stream(Estado.values())
56                  .filter(destino -> estado.permiteTransicionA(destino))
57                  .filter(destino -> !destino.requiereResponsable() || tieneResponsable())
58                  .toList();
59      }
        // ...
127     private void cambiarEstado() {
128         Optional<Solicitud> elegida = elegirSolicitudAbierta();
129         if (elegida.isEmpty()) {
130             return;
131         }
132         Solicitud solicitud = elegida.get();
133         if (!solicitud.tieneResponsable()) {
134             consola.mostrar("La solicitud #" + solicitud.getId() + " no tiene responsable asignado. "
135                     + "Solo puede rechazarse; para otros estados asignelo desde la opcion 3.");
136         }
137         consola.elegir("Nuevo estado:", solicitud.estadosPosibles(), Enum::name).ifPresent(nuevoEstado -> {
138             try {
139                 Solicitud actualizada = gestor.cambiarEstado(solicitud.getId(), nuevoEstado);
140                 consola.mostrar("Estado actualizado a " + actualizada.getEstado());
141             } catch (ReglaDeNegocioException ex) {
142                 mostrarError(ex);
143             }
144         });
145     }
```

CAP-61

CR-01 — después: el menú ofrece EN_PROCESO o RECHAZADA desde PENDIENTE, y RESUELTA o RECHAZADA desde EN_PROCESO (escenario E08).

CR-01 - el menú solo ofrece transiciones válidas — java tupad.ms2.Main (lo tipeado por el usuario, en amarillo)

```
Responsable asignado: Laura Gorosito

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | Laura Gorosito (Secretaria Academica)
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega Sánchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Nuevo estado:
1. EN_PROCESO
2. RECHAZADA
Opcion: 1
Estado actualizado a EN_PROCESO

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | EN_PROCESO | Laura Gorosito (Secretaria Academica)
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega Sánchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Nuevo estado:
1. RESUELTA
2. RECHAZADA
Opcion: 3
Opcion invalida
```

CAP-62

CR-02 — antes: `mostrarReporte()` nombraba cada estado y cada prioridad.

CR-02 · antes: el reporte nombraba cada estado y prioridad — Estado.java · MenuConsola.java

```
 3  public enum Estado {
 4      PENDIENTE,
 5      EN_PROCESO,
 6      RESUELTA,
 7      RECHAZADA;
        // ...
229      private void mostrarReporte() {
230          ResumenSolicitudes resumen = gestor.generarReporte();
231          consola.mostrar("===== REPORTE DE SOLICITUDES =====");
232          resumen.getSolicitudes().forEach(solicitud -> consola.mostrar(FormatoSolicitud.lineaReporte(solicitud)));
233          consola.mostrar("----------------------------------");
234          consola.mostrar("Total: " + resumen.getTotal());
235          consola.mostrar("Pendientes: " + resumen.getTotal(Estado.PENDIENTE)
236                  + " | En proceso: " + resumen.getTotal(Estado.EN_PROCESO)
237                  + " | Resueltas: " + resumen.getTotal(Estado.RESUELTA)
238                  + " | Rechazadas: " + resumen.getTotal(Estado.RECHAZADA));
239          consola.mostrar("Alta: " + resumen.getTotal(Prioridad.ALTA)
240                  + " | Media: " + resumen.getTotal(Prioridad.MEDIA)
241                  + " | Baja: " + resumen.getTotal(Prioridad.BAJA));
242      }
```

CAP-63

CR-02 — después: etiqueta plural en `Estado` y reporte que recorre `Estado.values()` y `Prioridad.values()`.

CR-02 · el reporte recorre los enums — Estado.java · MenuConsola.java

```
 3  public enum Estado {
 4      PENDIENTE("Pendientes"),
 5      EN_PROCESO("En proceso"),
 6      RESUELTA("Resueltas"),
 7      RECHAZADA("Rechazadas");
 8
 9      /** Nombre con el que se totaliza en el reporte, ej: "Pendientes: 2". */
10      private final String etiquetaPlural;
11
12      Estado(String etiquetaPlural) {
13          this.etiquetaPlural = etiquetaPlural;
14      }
15
16      public String getEtiquetaPlural() {
17          return etiquetaPlural;
18      }
        // ...
233      private void mostrarReporte() {
234          ResumenSolicitudes resumen = gestor.generarReporte();
235          consola.mostrar("===== REPORTE DE SOLICITUDES =====");
236          resumen.getSolicitudes().forEach(solicitud -> consola.mostrar(FormatoSolicitud.lineaReporte(solicitud)));
237          consola.mostrar("----------------------------------");
238          consola.mostrar("Total: " + resumen.getTotal());
239          // Se recorren los enums: un estado o una prioridad nueva aparece en el reporte sin tocar este método
240          consola.mostrar(Arrays.stream(Estado.values())
241                  .map(estado -> estado.getEtiquetaPlural() + ": " + resumen.getTotal(estado))
242                  .collect(Collectors.joining(" | ")));
243          consola.mostrar(Arrays.stream(Prioridad.values())
244                  .map(prioridad -> prioridad.getEtiqueta() + ": " + resumen.getTotal(prioridad))
245                  .collect(Collectors.joining(" | ")));
246      }
```

CAP-64

CR-03 — antes: una descripción de 5 espacios se aceptaba y se registraba la solicitud.

CR-03 · antes: una descripción de 5 espacios se aceptaba — java tupad.ms2.Main (lo tipeado por el usuario, en amarillo)

```
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 1
Descripcion: '     '
Prioridad (1=alta, 2=media, 3=baja): 1
Solicitud registrada con id 4
```

CAP-65

CR-03 — después: validación con `strip()` y casos nuevos en `SolicitudTest`.

CR-03 · la descripción se valida sin contar los espacios — Solicitud.java · SolicitudTest.java

```
28      public Solicitud(int id, Estudiante estudiante, NuevaSolicitud datos, LocalDate fecha) {
29          // Los espacios no cuentan: "     " no es una descripción
30          if (datos.descripcion() == null || datos.descripcion().strip().length() < LARGO_MINIMO_DESCRIPCION) {
31              throw new ReglaDeNegocioException("descripcion muy corta");
32          }
33          this.id = id;
34          this.estudiante = estudiante;
35          this.tipo = datos.tipo();
36          this.prioridad = datos.prioridad();
37          this.descripcion = datos.descripcion();
38          this.fecha = fecha;
39          this.estado = Estado.PENDIENTE;
40      }
        // ...
64          @ParameterizedTest(name = "descripción \"{0}\"")
65          @DisplayName("Rechaza descripciones nulas o de menos de 5 caracteres sin contar los espacios")
66          @NullSource
67          @ValueSource(strings = {"", "abc", "abcd", "     ", " ab ", "  abcd  ", "\t\t\t\t\t"})
68          void descripcionMuyCorta(String descripcion) {
69              ReglaDeNegocioException ex = assertThrows(ReglaDeNegocioException.class, () -> nuevaSolicitud(descripcion));
70              assertEquals("descripcion muy corta", ex.getMessage());
71          }
        // ...
73          @Test
74          @DisplayName("Acepta una descripción de exactamente 5 caracteres (valor límite)")
75          void descripcionEnElLimite() {
76              assertDoesNotThrow(() -> nuevaSolicitud("abcde"));
77              assertDoesNotThrow(() -> nuevaSolicitud("  abcde  "));
78          }
```

CAP-66

CR-03 — después: la misma entrada se rechaza con “descripcion muy corta”.

CR-03 · una descripción de 5 espacios se rechaza — java tupad.ms2.Main (lo tipeado por el usuario, en amarillo)

```
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 1
Descripcion: '     '
Prioridad (1=alta, 2=media, 3=baja): 1
Error: descripcion muy corta
```

CAP-67

CR-07 — antes: `registrarSolicitud()` con tres tareas y mensajes repetidos (30 líneas).

CR-07 · antes: registrarSolicitud() con tres tareas y mensajes repetidos — MenuConsola.java

```java
    private void registrarSolicitud() {
        List<Estudiante> estudiantes = gestor.getEstudiantes();
        if (estudiantes.isEmpty()) {
            consola.mostrar("No hay estudiantes cargados.");
            return;
        }
        Optional<Estudiante> estudiante = consola.elegir("Estudiante:", estudiantes, FormatoSolicitud::estudianteConEmail);
        if (estudiante.isEmpty()) {
            consola.mostrar("Registro cancelado.");
            return;
        }
        Optional<TipoSolicitud> tipo = consola.elegir("Tipo:", List.of(TipoSolicitud.values()), Enum::name);
        if (tipo.isEmpty()) {
            consola.mostrar("Registro cancelado.");
            return;
        }
        String descripcion = consola.leerLinea("Descripcion: ");
        String codigoPrioridad = consola.leerLinea("Prioridad (1=alta, 2=media, 3=baja): ");
        try {
            // Un texto no numerico ("ALTA") se rechaza igual que un numero fuera de rango (antes cerraba el programa)
            Prioridad prioridad = EntradaConsola.convertirANumero(codigoPrioridad)
                    .map(Prioridad::desdeCodigo)
                    .orElseThrow(() -> new ReglaDeNegocioException("prioridad invalida"));
            NuevaSolicitud datos = new NuevaSolicitud(estudiante.get().getLegajo(), tipo.get(), prioridad, descripcion);
            Solicitud registrada = gestor.registrarSolicitud(datos);
            consola.mostrar("Solicitud registrada con id " + registrada.getId());
        } catch (ReglaDeNegocioException ex) {
            mostrarError(ex);
        }
    }
```

CAP-68

CR-07 — después: `leerPrioridad()` extraído y mensajes como constantes (26 líneas).

CR-07 · leerPrioridad() extraído y mensajes como constantes — MenuConsola.java

```
30      private static final String OPCION_SALIR = "0";
31      private static final String SIN_ESTUDIANTES = "No hay estudiantes cargados.";
32      private static final String REGISTRO_CANCELADO = "Registro cancelado.";
        // ...
90      private void registrarSolicitud() {
91          List<Estudiante> estudiantes = gestor.getEstudiantes();
92          if (estudiantes.isEmpty()) {
93              consola.mostrar(SIN_ESTUDIANTES);
94              return;
95          }
96          Optional<Estudiante> estudiante = consola.elegir("Estudiante:", estudiantes, FormatoSolicitud::estudianteConEmail);
97          if (estudiante.isEmpty()) {
98              consola.mostrar(REGISTRO_CANCELADO);
99              return;
100         }
101         Optional<TipoSolicitud> tipo = consola.elegir("Tipo:", List.of(TipoSolicitud.values()), Enum::name);
102         if (tipo.isEmpty()) {
103             consola.mostrar(REGISTRO_CANCELADO);
104             return;
105         }
106         String descripcion = consola.leerLinea("Descripcion: ");
107         try {
108             Prioridad prioridad = leerPrioridad();
109             NuevaSolicitud datos = new NuevaSolicitud(estudiante.get().getLegajo(), tipo.get(), prioridad, descripcion);
110             Solicitud registrada = gestor.registrarSolicitud(datos);
111             consola.mostrar("Solicitud registrada con id " + registrada.getId());
112         } catch (ReglaDeNegocioException ex) {
113             mostrarError(ex);
114         }
115     }
        // ...
117     private Prioridad leerPrioridad() {
118         String codigo = consola.leerLinea("Prioridad (1=alta, 2=media, 3=baja): ");
119         // Un texto no numerico ("ALTA") se rechaza igual que un numero fuera de rango, sin cerrar el programa
120         return EntradaConsola.convertirANumero(codigo)
121                 .map(Prioridad::desdeCodigo)
122                 .orElseThrow(() -> new ReglaDeNegocioException("prioridad invalida"));
123     }
```

CAP-69

CR-10 — antes: comentarios que contaban la historia del código.

CR-10 · antes: comentarios que contaban la historia del código — DatosIniciales.java · NuevaSolicitud.java

```
14 /**
15  * Datos de ejemplo con los que arranca el sistema. Todos son sintéticos (no corresponden a personas reales).
16  * Antes estaban repartidos entre el constructor de GestorSolicitudes y Main.cargarDatosEjemplo().
17  */
18 public final class DatosIniciales {
        // ...
 6 /**
 7  * Datos que ingresa el usuario para registrar una solicitud (Parameter Object).
 8  * Agrupa en un solo objeto lo que antes viajaba como cuatro parámetros sueltos,
 9  * dos de ellos del mismo tipo y fáciles de intercambiar por error.
10  */
11 public record NuevaSolicitud(String legajo, TipoSolicitud tipo, Prioridad prioridad, String descripcion) {
12 }
```

CAP-70

CR-10 — después: comentarios que explican el porqué del diseño actual.

CR-10 · los comentarios explican el porqué — DatosIniciales.java · NuevaSolicitud.java

```java
/**
 * Datos de ejemplo con los que arranca el sistema. Todos son sintéticos (no corresponden a personas reales).
 * Están en una clase aparte para que Main solo arme el sistema y el gestor no dependa de datos fijos.
 */
public final class DatosIniciales {
    // ...
/**
 * Datos que ingresa el usuario para registrar una solicitud (Parameter Object).
 * Legajo y descripción son ambos String: como parámetros sueltos serían fáciles de
 * intercambiar por error; con nombre, cada dato queda identificado.
 */
public record NuevaSolicitud(String legajo, TipoSolicitud tipo, Prioridad prioridad, String descripcion) {
}
```

CAP-71

`gradlew clean check`: 88 tests en verde, verificación de cobertura mínima y BUILD SUCCESSFUL.

CAP-71 · gradlew clean check — 88 tests, 0 fallos; cobertura mínima verificada

```
> gradlew clean check

> Task :clean
> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
> Task :compileTestJava
> Task :processTestResources
> Task :testClasses
> Task :test
   ... (74 tests anteriores PASSED) ...
GestorSolicitudesTest > Buscar aplica el filtro recibido (estado, tipo, legajo y prioridad) PASSED
GestorSolicitudesTest > Un legajo inexistente se rechaza PASSED
GestorSolicitudesTest > Asignar un responsable inexistente se rechaza PASSED
GestorSolicitudesTest > Asignar responsable y avanzar el estado hasta RESUELTA PASSED
GestorSolicitudesTest > Un registro rechazado no consume id (regla del sistema base, escenario E19) PASSED
EntradaConsolaTest > elegir() numera las opciones desde 1 y devuelve la elegida PASSED
EntradaConsolaTest > convertirANumero() distingue números de textos PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta ""0"" PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta ""3"" PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta ""-1"" PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta ""dos"" PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta """" PASSED
EntradaConsolaTest > elegir() informa 'Opcion invalida' y devuelve vacío ante una respuesta fuera de la lista > respuesta "" 1"" PASSED
EntradaConsolaTest > leerLinea() muestra el mensaje en la misma línea y devuelve lo tipeado PASSED

> Task :jacocoTestCoverageVerification
> Task :check
> Task :jacocoTestReport

BUILD SUCCESSFUL in 5s
7 actionable tasks: 7 executed
```

CAP-72

Reporte de JaCoCo (`build/reports/jacoco/test/html/index.html`): 11 de 341 líneas y 3 de 93 ramas sin cubrir (96,8 % / 96,8 %; el HTML redondea las ramas a 96 %).

MS2_TPI_Grela_Higa — Sessions

MS2_TPI_Grela_Higa

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
tupad.ms2.ui		96%		91%	3	62	10	156	0	44	0	5
tupad.ms2		92%		n/a	1	2	1	10	1	2	0	1
tupad.ms2.datos		100%		100%	0	4	0	12	0	3	0	1
tupad.ms2.servicio		100%		100%	0	33	0	50	0	27	0	3
tupad.ms2.model		100%		100%	0	34	0	52	0	25	0	4
tupad.ms2.model.enums		100%		100%	0	23	0	34	0	12	0	3
tupad.ms2.repositorio		100%		100%	0	16	0	25	0	14	0	3
tupad.ms2.model.excepciones		100%		n/a	0	1	0	2	0	1	0	1
Total	28 of 2,021	98%	3 of 93	96%	4	175	11	341	1	128	0	21

Created with JaCoCo 0.8.14.202510111229

CAP-73

Reporte de pruebas de Gradle: 88 tests, 0 fallos, 100 % exitosos.

All Results

Gradle Test Run :test

Gradle Test Run :test

summary

88	0	0	2.349s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CaracterizacionConsolaTest	tupad.ms2.CaracterizacionConsolaTest	20	0	0	0.304s	100%
EstadoTest	tupad.ms2.model.EstadoTest	19	0	0	0.056s	100%
PrioridadTest	tupad.ms2.model.PrioridadTest	5	0	0	0.012s	100%
SolicitudTest	tupad.ms2.model.SolicitudTest	21	0	0	0.033s	100%
RepositoriosEnMemoriaTest	tupad.ms2.repositorio.RepositoriosEnMemoriaTest	5	0	0	0.004s	100%
GestorSolicitudesTest	tupad.ms2.servicio.GestorSolicitudesTest	9	0	0	0.011s	100%
EntradaConsolaTest	tupad.ms2.ui.EntradaConsolaTest	9	0	0	0.011s	100%

Generated by Gradle 9.3.0 at Nov 6, 2026, 12:53:15 AM

CAP-74

CR-12 — después: `asignarResponsable()` devuelve `void`; `Estudiante` y `Responsable` sin los getters sin uso.

CR-12 · sin código sin uso — GestorSolicitudes.java · Estudiante.java · Responsable.java

```
    // Asignar responsable (del listado) a solicitud
    public void asignarResponsable(int id, int idResponsable) {
        Solicitud solicitud = obtenerSolicitud(id);
        Responsable responsable = responsables.buscarPorId(idResponsable)
                .orElseThrow(() -> new ReglaDeNegocioException("responsable no encontrado"));
        solicitud.asignarResponsable(responsable);
    }
    // ...
```

```
public class Estudiante {
    private final String nombre;
    private final String legajo;
    private final String email;

    public Estudiante(String nombre, String legajo, String email) {
        this.nombre = nombre;
        this.legajo = legajo;
        this.email = email;
    }

    public String getNombre() {
        return nombre;
    }

    public String getLegajo() {
        return legajo;
    }

    public String getEmail() {
        return email;
    }
}
```

```
    // ...
public class Responsable {
    private final int id;
    private final String nombre;
    private final String area;

    public Responsable(int id, String nombre, String area) {
        this.id = id;
        this.nombre = nombre;
        this.area = area;
    }

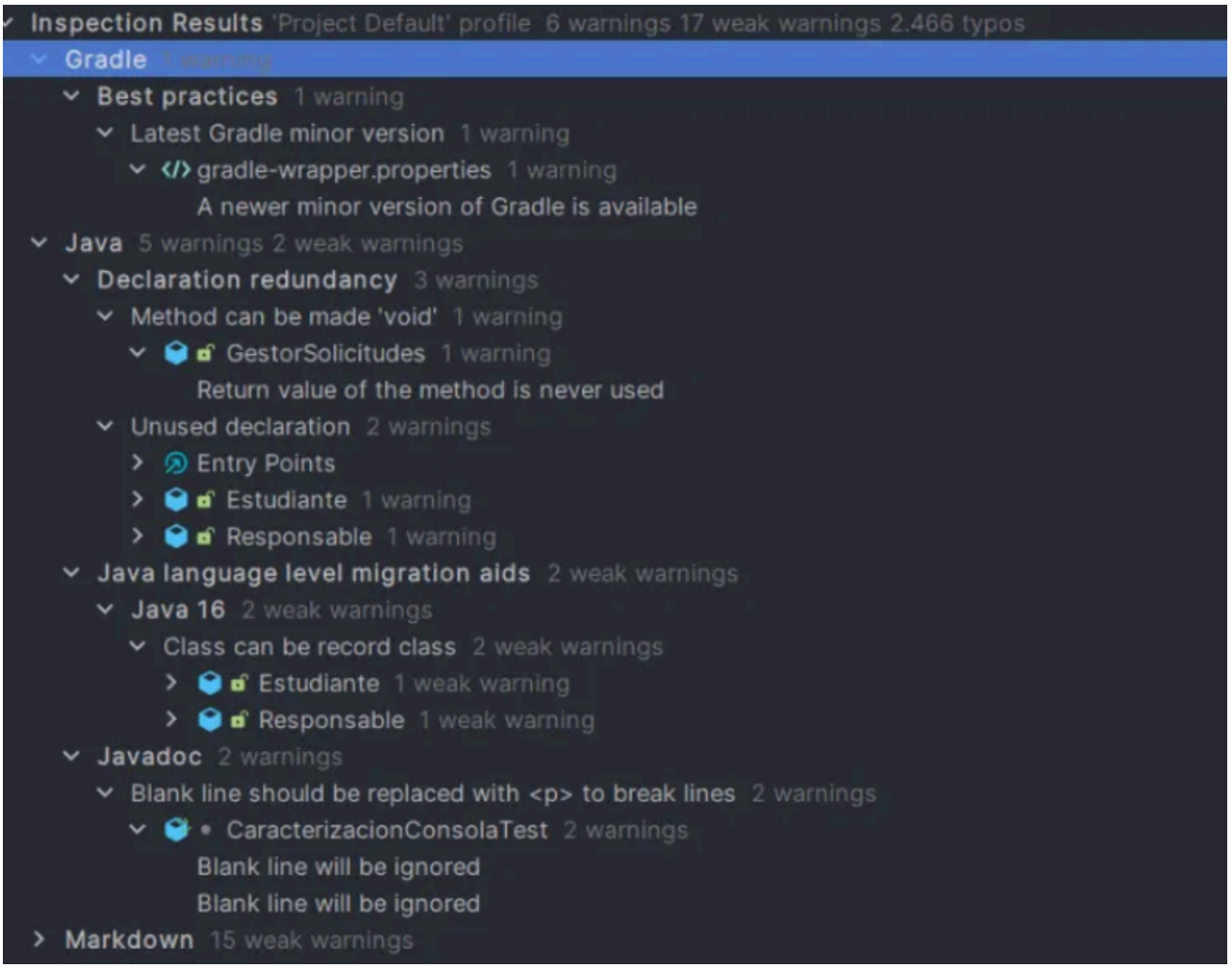
    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    // Formato con el que se muestra al responsable en toda la aplicacion, ej: "Laura Gorosito (Secretaria Academica)"
    public String getNombreConArea() {
        return nombre + " (" + area + ")";
    }
}
```

CAP-75

IntelliJ — Inspect Code sobre el proyecto (ítem C-07 del checklist, observación CR-12): 6 warnings, 17 weak warnings y typos del diccionario en inglés.



2. Uso de IA 🤖

2.1. Declaración general

Herramientas utilizadas	Claude (Anthropic): chat y Claude Code, un agente de programación que trabaja sobre la copia local del repositorio (lee y edita archivos, ejecuta comandos y pruebas). ChatGPT (OpenAI).
Finalidad general	Comprensión de conceptos, revisión de alternativas y mejora de redacción (etapas 1 a 3). Generación asistida de código, pruebas y documentación en las etapas 4 a 8, revisada y validada por el equipo.
Responsabilidad	El equipo revisó cada cambio antes de integrarlo, ejecutó las pruebas y es responsable de todo lo entregado. Las decisiones sobre las partes de cada integrante se acordaron entre ambos.

2.2 Escala de grado de intervención

Nivel	Descripción
Bajo	Consulta conceptual o aclaración de dudas. No se incorporó texto ni código generado.
Medio	Se pidieron alternativas o sugerencias. El equipo las analizó y adaptó; la decisión final fue propia.
Alto	Se incorporó contenido generado (texto o código) con modificaciones menores, revisado y validado por el equipo.

2.3 Registro por interacción

IA-01

Etapa	1 - Selección y descripción del sistema base
Herramienta	Claude
Objetivo de la consulta	Borrador del código para la versión inicial del sistema
Prompt	En relación con la consigna, el proyecto Java ya se encuentra creado en blanco y configurado con Gradle. Preciso que te enfoques en los puntos 3 y 4 de la consigna y que generes de manera individual una versión inicial de los archivos para el proyecto, incluyendo las entidades mínimas y los casos de uso mínimos allí especificados. Prioriza una implementación inicial y funcional, sin buscar una solución perfecta ni realizar refactorizaciones anticipadas, dado que posteriormente se deberán desarrollar los puntos restantes de la consigna.
Uso dado a la respuesta	Se tomó la estructura base propuesta por la IA: las entidades (Solicitud, Estudiante, Responsable y Estado), los cinco casos de uso (registrar, cambiar estado, asignar responsable, buscar/filtrar y reporte) y el menú de consola, que se adaptaron al paquete propio tupad.ms2.model. Se descartó la persistencia en archivo CSV (guardar() y cargar()) y se optó por memoria estructurada, que el enunciado admite y que simplifica el sistema base.
Grado de intervención	Medio
Evidencia	CAP-01

IA-02

Etapa	1 - Selección y descripción del sistema base
Herramienta	Claude
Objetivo de la consulta	Obtener una estructura base para el README
Prompt	Armame un template de README para el repositorio del proyecto, con descripción del sistema, requisitos, cómo compilar y ejecutar, y la estructura de carpetas
Uso dado a la respuesta	Se tomó la estructura de secciones propuesta y se completó con los datos reales del sistema. El contenido se actualizó en etapas posteriores (CR-04)
Grado de intervención	Medio
Evidencia	CAP-02

IA-03

Etapa	2 - Calidad
Herramienta	ChatGPT
Objetivo de la consulta	Crear un infograma sobre las calidades, para el documento
Prompt	Crea un infograma minimalista, tipo banner, muy resumido, sobre este párrafo:: [texto en sección “Sobre la calidad” en el documento]
Uso dado a la respuesta	infograma para el documento
Grado de intervención	Alto
Evidencia	CAP-03

IA-04

Etapa	2 - Calidad
Herramienta	Claude
Objetivo de la consulta	Métricas de UX para calidad externa
Prompt	Dame un template de metricas de UX para el informe de calidad externa, con datos ficticios pero que reflejen la calidad actual del software.
Uso dado a la respuesta	Tablas de métricas
Grado de intervención	Alto
Evidencia	CAP-04

IA-05

Etapa	3 - Code Smells
Herramienta	Claude
Objetivo de la consulta	Listar code smells
Prompt	En la Etapa 3 tenemos que identificar al menos 8 code smells o debilidades técnicas en nuestro sistema base (gestión de solicitudes académicas, en Java, con consola y persistencia en memoria). Te adjunto el código de la versión estado-inicial. Necesitamos solo una lista de los code smells que detectes, con este formato por cada uno: -Nombre del smell (según el catálogo de Fowler, si corresponde) -Archivo y método donde aparece -Una frase breve que indique qué lo evidencia
Uso dado a la respuesta	Resumen y listado
Grado de intervención	Alto
Evidencia	CAP-56

IA-06

Etapa	3 - Code Smells
Herramienta	Claude
Objetivo de la consulta	Simplificación de las ubicaciones en el informe
Prompt	Es realmente necesario señalar todas las líneas de código en todos los casos dentro de cada code smell?
Uso dado a la respuesta	Aclaración
Grado de intervención	Bajo
Evidencia	CAP-57

IA-07

Etapa	3 - Code Smells
Herramienta	Claude
Objetivo de la consulta	Chequeo gral de etapa
Prompt	Terminada la etapa 3, revisa como quedó el proyecto
Uso dado a la respuesta	Correcciones varias
Grado de intervención	Medio
Evidencia	CAP-58

IA-08

Etapa	4 a 6 - Refactoring seguro y pruebas
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Planificar e implementar el refactoring seguro y las pruebas de caracterización sobre el diagnóstico de la Etapa 3
Prompt	“Con los code smells de la Etapa 3, armemos un plan para las etapas 4 a 6: primero las pruebas de caracterización que congelen el comportamiento, después los refactorings de a uno por commit, corriendo la suite después de cada uno”
Uso dado a la respuesta	Se siguió el plan propuesto: pruebas de caracterización (golden master) y luego 10 refactorings, uno por commit, en la rama refactor/etapa-4-5 (PR #3). Ariel revisó cada cambio y ejecutó la suite después de cada refactoring. Las pruebas detectaron una regresión en R-08 en el código generado, que se corrigió
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo) e historial de commits del repositorio

IA-09

Etapa	4 a 7 - Repositorio
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Comprender qué son los tags de Git y para qué los pide la consigna.
Prompt	“Me explicas bien los tags?”
Uso dado a la respuesta	Comprensión del concepto para organizar las entregas. No se incorporó contenido generado.
Grado de intervención	Bajo
Evidencia	Exportación de la conversación (ZIP del anexo) e historial de commits del repositorio

IA-10

Etapa	4 a 7 - Repositorio
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Publicar el trabajo por etapas: una rama y un Pull Request por etapa, revisado por el otro integrante, sin incluir el informe.
Prompt	“Quiero reoriganizar las ramas una por una , generame orden para poder pushearlo correctamente sin perder el orden.”
Uso dado a la respuesta	La IA reorganizó las ramas, volvió a aplicar los commits de cada rama el día de su publicación y ejecutó los push. Cada Pull Request lo revisó y mergeó el otro integrante.
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo) e historial de commits del repositorio

IA-11

Etapa	7 - Code Review
Herramienta	Claude
Objetivo de la consulta	Planificar la Etapa 7, definir la estructura del checklist técnico y revisar el código en busca de observaciones
Prompt	“Tenemos que hacer el code review de la Etapa 7. Ayudame a planificarlo y a armar la estructura de un checklist técnico. Después revisemos el código y las pruebas buscando observaciones ”
Uso dado a la respuesta	Se usó como base para el plan de la etapa y para las secciones 1 a 6 de docs/checklist.md. Matías verificó cada observación en el código y en la ejecución antes de registrarla. Los IDs, la severidad y qué aceptar como deuda se definieron con Ariel
Grado de intervención	Medio
Evidencia	Exportación de la conversación (ZIP del anexo)

IA-12

Etapa	7 - Code review
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Completar las secciones 6 a 8 del checklist (resultados de pruebas, repositorio y datos) siguiendo el plan de trabajo acordado con Matías.
Prompt	“Dale, arranquemos la fase B” (con el plan de pasos de Matías como contexto).
Uso dado a la respuesta	Se cargaron los resultados de `gradlew check`, la evaluación de los ítems GH-01 a GH-06, S-01 y S-02, la actualización de CR-04 y la observación nueva CR-13 (commit `690f368`). Ariel revisó el resultado antes de autorizar el commit y el push.
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo) e historial de commits del repositorio

IA-13

Etapa	7 - Code review
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Implementar las correcciones del code review y completar la documentación técnica en /docs.
Prompt	“Corregí las observaciones CR-01, CR-02, CR-03, CR-07, CR-10 y CR-12 del checklist, un commit por observación, y corré la suite después de cada una. Después completá /docs con el plan de pruebas, el registro de refactorings y las decisiones técnicas (CR-13)”
Uso dado a la respuesta	Se aplicaron las correcciones en commits separados (58ece6b, 85478a0, 771d9cf, b4a59b2, 5849655, c8daacb y 70fb575). Ariel revisó cada diff y la ejecución de la suite. Los cambios de salida de CR-01 se justificaron en los goldens E07, E08 y E18.
Grado de intervención	Alto
Evidencia	Historial de commits del repositorio

IA-14

Etapa	8 - Conclusiones
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Redactar la conclusión individual de Ariel a partir del trabajo realizado en conjunto.
Prompt	“Emprolija las conclusiones”.
Uso dado a la respuesta	Se incorporó como conclusión individual de Ariel en `docs/conclusiones.md` y en el informe, después de que él la revisara.
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo) e historial de commits del repositorio

IA-15

Etapa	8 - Conclusiones
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Obtener un borrador del informe final y del anexo a partir del repositorio y la documentación de cada etapa
Prompt	“Con lo que hay en el repositorio y en /docs, armá un borrador del informe final siguiendo la estructura de la consigna, y un anexo con las capturas y el registro de uso de IA ”.
Uso dado a la respuesta	Se usó como punto de partida. El equipo revisó y reescribió las secciones, verificó los datos contra el repositorio y completó las capturas y las conclusiones individuales.
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo)

IA-16

Etapa	8 - Conclusiones
Herramienta	Claude (Claude Code)
Objetivo de la consulta	Redactar la conclusión individual de Matias a partir del trabajo realizado en conjunto.
Prompt	“Armá un borrador de mi conclusión individual: me tocaron las etapas 1 a 3, el README y en la Etapa 7 el checklist, el review del código y las pruebas, y las correcciones CR-04, CR-05 y CR-06 ”.
Uso dado a la respuesta	Matías revisó el borrador y lo ajustó con su experiencia personal: el problema de la tilde en la ruta, la corrección incompleta del README y el uso de IA en la Etapa 7. Se incorporó en docs/conclusiones.md y en el informe
Grado de intervención	Alto
Evidencia	Exportación de la conversación (ZIP del anexo)

3. Bibliografía y recursos consultados

- Code Smells https://refactoring.guru/refactoring/smells
- Refactoring Guru — Catálogo de técnicas de refactoring. https://refactoring.guru/refactoring/techniques
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2.ª ed.). Addison-Wesley.
- Feathers, M. (2004). Working Effectively with Legacy Code. Prentice Hall (pruebas de caracterización).
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- JUnit — User Guide. https://junit.org/
- JaCoCo — Java Code Coverage Library. https://www.jacoco.org/jacoco/
- Conventional Commits 1.0.0. https://www.conventionalcommits.org/es/v1.0.0/
- Git — gitattributes (finales de línea). https://git-scm.com/docs/gitattributes
- RFC 2606 — Reserved Top Level DNS Names (dominios de ejemplo). https://www.rfc-editor.org/rfc/rfc2606
- Material de la cátedra Metodología de Sistemas II, Unidades 1 a 5.